

PW 2: DATA WAREHOUSE REPORT

Data Models in Database Systems, 2025

Hospital Patient Admission

Group Members:

- Ngoc Bao Tram Tran, 231ADB294
- Ilaha Habibova, 231ADB231

Introduction

This report documents the design, implementation, and analysis of a data warehouse for the **Hospital Patient Admission** domain. The primary goal of this project was to interpret the provided business requirements to build a star schema capable of answering key analytical questions.

We designed our schema to support analyses such as “analyzing admission/ readmission”; “tracking length of stay”, “analyzing physician performance and workload”. This document details our design choices, implementation process, and the results of our analytical queries.

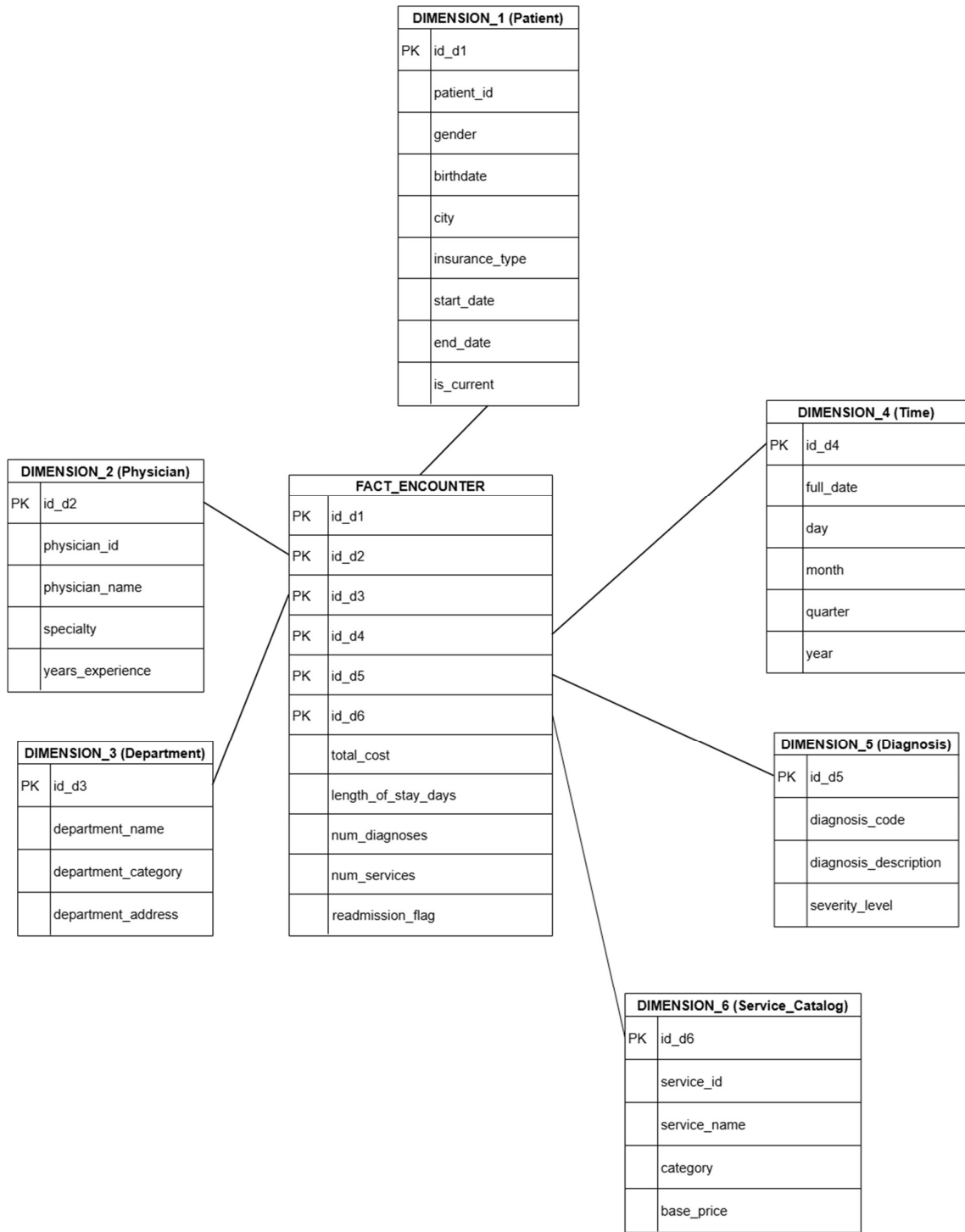
1. Task 1: Design the Data Warehouse

This section covers our final data warehouse design and the justifications for our decisions.

Goal of the Data Warehouse: To support hospital-wide analytics including patient flow, physician workload, readmission tracking, treatment cost analysis, and time-based performance trends for data-driven decision-making.

1.1 ER Diagram

Below is the Entity-Relationship (ER) diagram for our star schema.



1.2. Design Justification

We have designed the data warehouse using Star Schema, since it is the most appropriate modelling approach when considering analytical workloads in the hospital domain.

A star schema provides:

- Simple, fast query performance, ideal for BI dashboards and aggregated hospital statistics.
- Clear Distinction between facts (events) and dimensions (descriptive context)
- Easy support for OLAP operations such as roll-up, drill-down, slice, and dice
- Straightforward integration of SCD Type 2, which is significant for tracking historical patient information.

Hospitals usually perform analytical queries on a large amount of data, for example, readmission trends, physician workload analysis, department performance, or treatment cost analyses. A star schema will reduce joins and optimize the performance for this kind of operation.

Fact Table: FACT_ENCOUNTER

- **Grain:**

Each row in FACT_ENCOUNTER represents one hospital encounter - one patient admission event.

This is the lowest level of detail that we want to analyse. All the analytical metrics-length of stay, number of services, readmission status, and total cost-are all bound to this single encounter.

- **Measures:**

- total_cost - Total treatment cost for this encounter (calculated from services performed)
- length_of_stay_days - Number of days the patient stayed in the hospital
- num_diagnoses - Number of diagnoses were assigned during the encounter
- num_services - Number of medical services performed during the encounter
- readmission_flag - 'Y' if patient was readmitted within 30 days, else 'N'

Dimension Tables

DIMENSION_1 — Patient

Explanation:

This dimension contains descriptive information of all patients. It also involves Slowly Changing Dimension Type 2, whereby the warehouse will retain the history in case a patient's city of residence, insurance type, or any of the other attributes changes. This is important for correct analytics on readmission trends, impacts of insurance, and demographic-based cost studies.

Key Attributes

- id_d1 - surrogate key of fact table, which distinguishes uniquely each version of the patient record.
- patient_id - patient's personal id
- gender, birth date - used in demographic and medical analysis, such as cost per age group.
- city - may evolve with time, necessary for geographic analysis.
- insurance_type - defines the cost coverage level and affects readmission patterns.
- start_date, end_date, is_current – SCD Type 2 fields to track the validity period of each historic version.

Justification of Key Attributes

These attributes enable the hospital to analyse patients by demographics, track insurance-based performance, and to view historical data correctly at the time of each encounter. It is important that these changes in data are stored for answering questions like:

- Did patients with different demographic details (such as age groups derived from their birthdates [function - get_age_group], or different cities) have different readmission rates?
- Did changes in insurance type affect the total treatment cost?

DIMENSION_2 — Physician

Explanation:

This dimension describes physicians involved in patient encounters. It enables analysis of medical staff performance, workload, experience level, and specialty trends.

Key Attributes

- id_d2 – surrogate key linking to the fact table
- physician_id – physician's id in the hospital
- physician_name – used in reporting
- specialty – determines which department the physician belongs to
- years_experience: supports performance and outcome analysis by seniority

Justification of Key Attributes

Physicians are central for analytics in hospitals. These attributes enable: workload calculation, treatment cost analysis by physician, performance comparison across specialties, identifying which specialists handle more severe cases.

DIMENSION_3 — Department

Explanation:

Stores descriptive information about each hospital department, such as Cardiology, Orthopedics. The dimension enables analysis of performance at the departmental level.

Key Attributes

- id_d3 – surrogate key
- department_name – principal reporting attribute utilized in almost all queries
- department_category – categorizes departments, e.g. Surgical, Medical
- department_address – useful for multi-location hospitals or wards

Justification of Key Attributes

Departments constitute an important analytical axis. These attributes help answer questions like:

- Which department has the highest treatment costs?
- Which departments have the highest readmission rate?
- What are the monthly/quarterly trends for each department?

DIMENSION_4 — Time

Explanation:

This dimension stores calendar information about each encounter date. It supports OLAP time hierarchies, such as roll-up and drill-down.

Key Attributes

- id_d4 – surrogate key
- full_date – exact date of encounter
- day, month, quarter, year - time hierarchy attributes

Justification of Key Attributes

The time-based attributes are crucial in health analysis. These attributes support

- Monthly Readmission Trend Analysis
- Quarterly cost analysis
- Comparisons of annual physician workload

The hierarchy Day → Month → Quarter → Year allows flexible reporting at different granularities, which supports roll-up analysis.

DIMENSION_5 — Diagnosis

Explanation:

Describes medical diagnoses assigned during an encounter.

Key Attributes

- id_d5 – surrogate key
- diagnosis_code - code of each diagnosis used in hospital
- diagnosis_description – meaningful text description
- severity_level – medical severity used in risk analysis

Justification of Key Attributes

These attributes allow the hospital to understand:

- which diagnoses have higher readmission rates
- the relationship between severity level and patient outcome

DIMENSION_6 — Service Catalog

Explanation:

Includes the medical services provided, such as X-Rays, MRI, surgeries, etc. It is used in assessing the total cost of any encounter and analyzing the distribution of hospital services provided.

Key Attributes

- id_d6 – surrogate key
- service_id – business identifier
- service_name – readable description
- category – service grouping (imaging, lab, surgery, etc.)
- base_price - used to calculate and compare expected vs. actual cost

Justification of Key Attributes

These attributes support :

- cost analysis by type of service comparing physicians or departments in terms of service utilization
- validating cost vs. baseline price which is important for audit and budgeting purposes

(Optional) Slowly Changing Dimensions (SCD)

SCD Type 2 was implemented for DIMENSION_1, which represents the Patient.

Why this dimension needs SCD2:

Changes to patient information historically, including:

- city relocation
- insurance_type
- insurance provider

These changes must not overwrite historical data because:

- cost analysis by insurance must reflect the correct insurance at the time
- readmission patterns differ by insurance type.
- patients may relocate to another city, thereby influencing resource allocation

How SCD Type 2 works in our schema:

- patient_id is the business key, real individual
- id_d1 is a surrogate key, one per version

When patient data changes:

- old record is updated: end_date set, is_current = 'N'
- new record inserted: start_date updated, is_current = 'Y'

This lets us observe which version of patient information was active during each encounter trends before/after insurance or demographic changes.

2. Task 2: Implement the Warehouse (ETL)

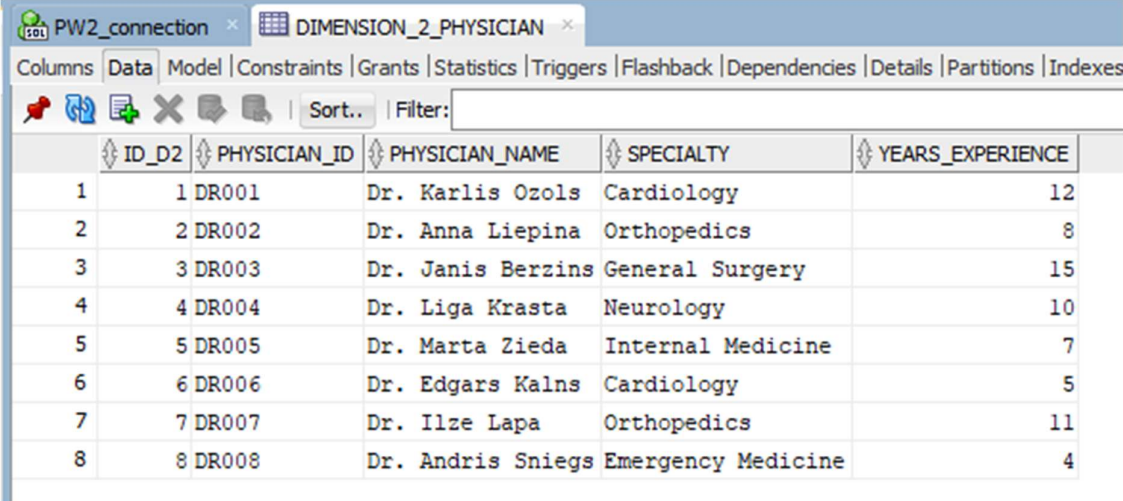
In our implementation, we created and populated six dimension tables and one central fact table. The dimension tables DIMENSION_1_PATIENT, DIMENSION_2_PHYSICIAN, DIMENSION_3_DEPARTMENT, DIMENSION_4_TIME, DIMENSION_5_DIAGNOSIS, and DIMENSION_6_SERVICE_CATALOG store the descriptive context for analysis: patient demographics with SCD Type 2 history, physician information and specialty with their years of experience, department details (address and category), the calendar hierarchy (day-month-quarter-year) with full date format, diagnosis codes and descriptions with severity levels, and the hospital service catalog (service category and name) with base prices.

The fact table FACT_ENCOUNTER records individual hospital encounters. Each row links to all six dimensions via surrogate keys and contains the main analytical measures such as total_cost, length_of_stay_days, num_diagnoses, num_services, and readmission_flag. All dimension tables were populated with realistic sample data using INSERT ... SELECT statements, and the fact table was filled by the PL/SQL procedure load_fact_encounter, which generates 60 consistent encounter records each time the procedure is executed, based on the existing dimension data.

Table DIMENSION_1_PATIENT

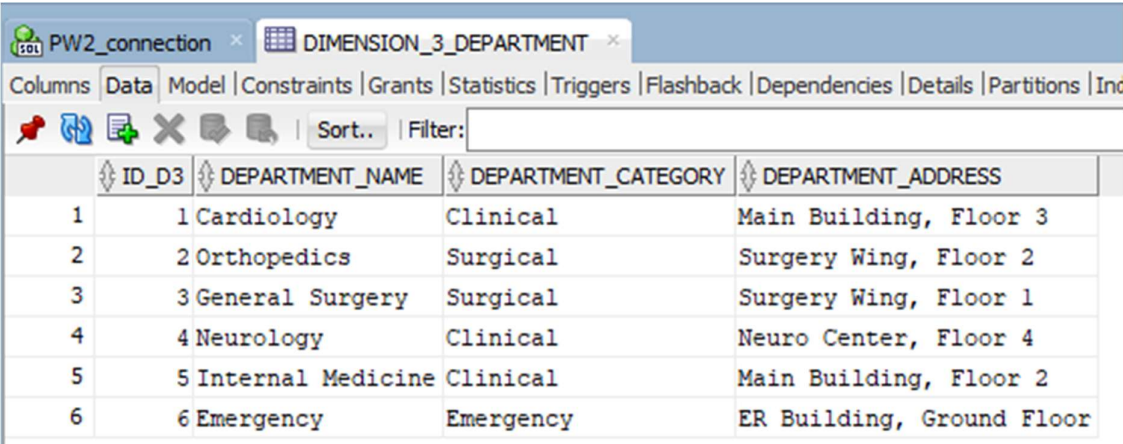
ID_D1	PATIENT_ID	PATIENT_NAME	GENDER	BIRTHDATE	CITY	INSURANCE_TYPE	START_DATE	END_DATE	IS_CURRENT
1	1 P001	Anna Ozola	F	10-MAR-85	Riga	Public	01-JAN-18	31-DEC-19	N
2	2 P002	Janis Kalns	M	25-AUG-70	Liepaja	Private	01-JAN-15	31-DEC-18	N
3	3 P003	Liga Berzina	F	15-NOV-92	Jelgava	Public	10-FEB-21	31-DEC-99	Y
4	4 P004	Marta Vilcane	F	02-JUN-65	Riga	Private	01-JUL-18	31-DEC-99	Y
5	5 P005	Edgars Liepa	M	20-JAN-99	Ventspils	Public	01-MAR-22	31-DEC-99	Y
6	6 P006	Sandra Krume	F	09-SEP-88	Daugavpils	Private	01-SEP-20	31-DEC-99	Y
7	7 P007	Arturs Ozols	M	30-DEC-78	Riga	Public	01-JAN-17	31-DEC-99	Y
8	8 P008	Ilze Jansone	F	14-MAY-01	Liepaja	Private	15-JAN-23	31-DEC-99	Y
9	9 P009	Karlis Romanovs	M	22-APR-80	Riga	Public	01-JAN-15	31-DEC-20	N
10	10 P010	Elina Staltere	F	19-JUL-95	Jurmala	Private	01-JUN-18	15-JUL-21	N
11	11 P001	Anna Ozola	F	10-MAR-85	Riga	Public	01-JAN-18	31-DEC-99	Y
12	12 P002	Janis Kalns	M	25-AUG-70	Liepaja	Public	01-JAN-15	31-DEC-99	Y
13	13 P009	Karlis Romanovs	M	22-APR-80	Riga	Private	01-JAN-21	31-DEC-99	Y

Table DIMENSION_2_PHYSICIAN



	ID_D2	PHYSICIAN_ID	PHYSICIAN_NAME	SPECIALTY	YEARS_EXPERIENCE
1	1	DR001	Dr. Karlis Ozols	Cardiology	12
2	2	DR002	Dr. Anna Liepina	Orthopedics	8
3	3	DR003	Dr. Janis Berzins	General Surgery	15
4	4	DR004	Dr. Liga Krasta	Neurology	10
5	5	DR005	Dr. Marta Zieda	Internal Medicine	7
6	6	DR006	Dr. Edgars Kalns	Cardiology	5
7	7	DR007	Dr. Ilze Lapa	Orthopedics	11
8	8	DR008	Dr. Andris Sniegs	Emergency Medicine	4

Table DIMENSION_3_DEPARTMENT



	ID_D3	DEPARTMENT_NAME	DEPARTMENT_CATEGORY	DEPARTMENT_ADDRESS
1	1	Cardiology	Clinical	Main Building, Floor 3
2	2	Orthopedics	Surgical	Surgery Wing, Floor 2
3	3	General Surgery	Surgical	Surgery Wing, Floor 1
4	4	Neurology	Clinical	Neuro Center, Floor 4
5	5	Internal Medicine	Clinical	Main Building, Floor 2
6	6	Emergency	Emergency	ER Building, Ground Floor

Table DIMENSION_4_TIME

PW2_connection x DIMENSION_4_TIME x						
Columns Data Model Constraints Grants Statistics Triggers Flashback De						
Sort.. Filter:						
	ID_D4	FULL_DATE	DAY	MONTH	QUARTER	YEAR
1	1	05-JAN-25	5	1	1	2025
2	2	10-JAN-25	10	1	1	2025
3	3	20-JAN-25	20	1	1	2025
4	4	05-FEB-25	5	2	1	2025
5	5	18-FEB-25	18	2	1	2025
6	6	02-MAR-25	2	3	1	2025
7	7	15-MAR-25	15	3	1	2025
8	8	01-APR-25	1	4	2	2025
9	9	10-APR-25	10	4	2	2025
10	10	05-MAY-25	5	5	2	2025

Table DIMENSION_5_DIAGNOSIS

PW2_connection		DIMENSION_5_DIAGNOSIS	
Columns	Data	Model	Constraints Grants Statistics Triggers Flashback Dependencies Details Partitions
		Sort..	Filter:
	ID_D5	DIAGNOSIS_CODE	DIAGNOSIS_DESCRIPTION
1	1	I21	Acute myocardial infarction
2	2	I10	Essential (primary) hypertension
3	3	M16	Osteoarthritis of hip
4	4	M48	Other spondylopathies
5	5	J18	Pneumonia, unspecified organism
6	6	E11	Type 2 diabetes mellitus
7	7	K35	Acute appendicitis
8	8	S72	Fracture of femur

Table DIMENSION_6_SERVICE_CATALOG


```

-- The subquery builds combinations of valid dimension rows and
-- generates realistic random measures for analysis.

INSERT INTO FACT_ENCOUNTER (
    id_d1,
    id_d2,
    id_d3,
    id_d4,
    id_d5,
    id_d6,
    total_cost,
    length_of_stay_days,
    num_diagnoses,
    num_services,
    readmission_flag
)
SELECT *
FROM (
    SELECT
        -- Foreign keys from the dimension tables
        p.id_d1,  -- patient (DIMENSION_1_PATIENT)
        ph.id_d2, -- physician (DIMENSION_2_PHYSICIAN)
        d.id_d3,  -- department (DIMENSION_3_DEPARTMENT)
        t.id_d4,  -- time (DIMENSION_4_TIME)
        dg.id_d5, -- diagnosis (DIMENSION_5_DIAGNOSIS)
        s.id_d6,  -- service (DIMENSION_6_SERVICE_CATALOG)

        -- Create measures with some randomness
        -- TRUNC() is used to truncate the random number to an integer
        (s.base_price * TRUNC(DBMS_RANDOM.VALUE(1,4))) AS total_cost,
        -- total_cost = base price × random multiplier between 1–3

        TRUNC(DBMS_RANDOM.VALUE(1,15)) AS length_of_stay_days,
        -- random length of stay between 1 and 14 days

        TRUNC(DBMS_RANDOM.VALUE(1,4)) AS num_diagnoses,
        -- random number of diagnoses between 1 and 3

        TRUNC(DBMS_RANDOM.VALUE(1,5)) AS num_services,
        -- random number of services between 1 and 4

        -- Readmission indicator
        CASE
            WHEN DBMS_RANDOM.VALUE(0,1) > 0.8 THEN 'Y'
            ELSE 'N'
        END AS readmission_flag
        -- ~20% of admissions are marked as readmissions

    -- Join logic
    FROM DIMENSION_1_PATIENT p
    JOIN DIMENSION_4_TIME t
        ON t.full_date BETWEEN p.start_date AND p.end_date
        -- Only generate encounters during the active SCD period of the patient

    JOIN DIMENSION_2_PHYSICIAN ph
        ON 1 = 1 -- behaves as a CROSS JOIN (all doctors available)

    JOIN DIMENSION_3_DEPARTMENT d
        ON ( (ph.specialty = 'Cardiology' AND d.department_name = 'Cardiology')
            OR (ph.specialty = 'Orthopedics' AND d.department_name = 'Orthopedics')

```

```

    OR (ph.specialty = 'Neurology'      AND d.department_name = 'Neurology')
    OR (ph.specialty = 'Internal Medicine' AND d.department_name = 'Internal Medicine')
    OR (ph.specialty = 'Emergency Medicine' AND d.department_name = 'Emergency')
  )
  -- Ensures each doctor is assigned only to the matching department

JOIN DIMENSION_5_DIAGNOSIS dg
ON ( (d.department_name = 'Cardiology' AND dg.diagnosis_code IN ('I21','I10'))
    OR (d.department_name = 'Orthopedics' AND dg.diagnosis_code IN ('M16','S72'))
    OR (d.department_name = 'General Surgery' AND dg.diagnosis_code IN ('K35','S72'))
    OR (d.department_name = 'Neurology' AND dg.diagnosis_code IN ('M48'))
    OR (d.department_name = 'Internal Medicine' AND dg.diagnosis_code IN ('E11','J18','I10'))
    OR (d.department_name = 'Emergency' AND dg.diagnosis_code IN ('I21','J18','K35','S72'))
  )
  -- Restrict diagnosis to realistic options for each department

JOIN DIMENSION_6_SERVICE_CATALOG s
ON (
  -- Cardiology: ECG, imaging, blood tests
  (d.department_name = 'Cardiology'
   AND s.service_name IN ('ECG','MRI Scan','CT Scan','Blood Test Panel'))

  OR -- Orthopedics: imaging, surgery, physio
  (d.department_name = 'Orthopedics'
   AND s.service_name IN ('X-Ray','MRI Scan',
                        'Hip Replacement Surgery','Physiotherapy Session'))

  OR -- General Surgery: appendectomy, imaging, blood tests
  (d.department_name = 'General Surgery'
   AND s.service_name IN ('Appendectomy','CT Scan','X-Ray','Blood Test Panel'))

  OR -- Neurology: imaging, blood tests
  (d.department_name = 'Neurology'
   AND s.service_name IN ('MRI Scan','CT Scan','Blood Test Panel'))

  OR -- Internal Medicine: imaging + basic tests + ECG
  (d.department_name = 'Internal Medicine'
   AND s.service_name IN ('MRI Scan','CT Scan','X-Ray','Blood Test Panel','ECG'))

  OR -- Emergency: almost everything
  (d.department_name = 'Emergency'
   AND s.service_name IN ('MRI Scan','CT Scan','X-Ray',
                        'Blood Test Panel','ECG',
                        'Appendectomy','Physiotherapy Session'))
  )

WHERE p.is_current = 'Y'
  -- Only use current SCD versions for generating encounters

ORDER BY DBMS_RANDOM.VALUE
  -- Shuffle rows so the first 60 chosen by ROWNUM are random
) src
WHERE ROWNUM <= 60;
  -- Limit output to 60 fact rows

COMMIT;
  -- Save changes to the database
END;
```

EXEC load_fact_encounter;

Procedure Explanation:

The load_fact_encounter procedure will automatically generate realistic hospital encounter records to the fact table based on existing dimension data. Each execution creates sixty new encounters by matching valid patients, physicians, departments, diagnoses, services, and dates. The joins ensure that each encounter makes sense: doctors are matched with the proper department based on their specialty, and each department is only matched with diagnoses and services that are medically appropriate. Random values are also added for cost, length of stay, number of diagnoses, and number of services to make the data more diverse and realistic. Finally, the selected records are inserted into the fact table, creating a complete and analyzable set of encounter data without manually entering anything.

3. Task 3: Advanced Analytical Queries

3.1 ROLLUP/CUBE Queries

- **Query 1: CUBE** - What is the average length of stay for a patient, and how does it differ by department or primary diagnosis
- **Business Question:** What is the average length of stay for hospital encounters overall, and how does this average vary across departments and primary diagnoses?
- **SQL Query:**

```
-- Calculate the average length of stay by department, by diagnosis,
-- by the combination of department + diagnosis, and for all encounters.
SELECT
    d.department_name,                -- Hospital department (e.g., Cardiology, Orthopedics)
    dg.diagnosis_description AS primary_diagnosis, -- Primary diagnosis description
    AVG(f.length_of_stay_days) AS avg_length_of_stay -- Average number of days per group
FROM FACT_ENCOUNTER f
JOIN DIMENSION_3_DEPARTMENT d
    ON f.id_d3 = d.id_d3                -- Link each encounter to its department
JOIN DIMENSION_5_DIAGNOSIS dg
    ON f.id_d5 = dg.id_d5                -- Link each encounter to its primary diagnosis
-- Use CUBE to generate all combinations of the two dimensions:
-- 1) department + diagnosis
-- 2) department only
-- 3) diagnosis only
-- 4) grand total (all departments & diagnoses)
GROUP BY CUBE (d.department_name, dg.diagnosis_description)
-- Sort output by department name and diagnosis description ascendingly
ORDER BY d.department_name, dg.diagnosis_description;
```

- **Technical Explanation:** This query uses CUBE (d.department_name, dg.diagnosis_description) to calculate the average length of stay at several levels of aggregation. It joins the FACT_ENCOUNTER table with dimension tables about the

department (DIMENSION_3_DEPARTMENT) and diagnosis (DIMENSION_5_DIAGNOSIS), then calculates AVG(f.length_of_stay_days) for:

- each (department, diagnosis) pair,
- each department across all diagnoses,
- each diagnosis across all departments,
- and the overall hospital average across all encounters.

- **Query 2: ROLLUP** - Analyze physician workload by looking at the number of patients they admit, and compare treatment costs across different doctors for the same diagnosis.
- **Business Question:** How do the different physicians compare for each diagnosis with respect to their total treatment cost, average treatment cost, and workload (number of patients they admit)? How does each doctor's average cost compare to the overall average cost for that diagnosis?
- **SQL Query:**

```
SELECT
    dg.diagnosis_description,
    ph.physician_name,

    -- Total treatment cost per doctor for each diagnosis
    SUM(f.total_cost) AS total_cost,

    -- Average treatment cost per doctor for each diagnosis
    AVG(f.total_cost) AS average_cost,

    -- Workload: total number of patients' admissions handled by the doctor
    COUNT(*) AS total_admissions,

    -- Diagnosis-level: -
    -- average treatment cost across all doctors for this diagnosis
    --
    AVG(AVG(f.total_cost)) OVER (
        PARTITION BY dg.diagnosis_description
    ) AS diagnosis_average_cost,

    -- Comparison label 'cost_position_within_diagnosis'
    -- indicates whether this doctor's average cost is above,
    -- below, or equal to the diagnosis-level average

    -- If this doctor's average treatment cost for the diagnosis
    -- is greater than the overall average cost for that diagnosis
    CASE
        WHEN AVG(f.total_cost) >
            AVG(AVG(f.total_cost)) OVER (PARTITION BY dg.diagnosis_description)
        THEN 'Above diagnosis average'
    -- If this doctor's average treatment cost for the diagnosis
    -- is less than the overall diagnosis-level average cost
    WHEN AVG(f.total_cost) <
```

```

        AVG(AVG(f.total_cost)) OVER (PARTITION BY dg.diagnosis_description)
        THEN 'Below diagnosis average'
    -- If neither greater nor less (i.e., equal or very close),
    -- we label it as being at the diagnosis average
        ELSE 'At diagnosis average'
    END AS cost_position_within_diagnosis

FROM FACT_ENCOUNTER f
JOIN DIMENSION_5_DIAGNOSIS dg ON f.id_d5 = dg.id_d5
JOIN DIMENSION_2_PHYSICIAN ph ON f.id_d2 = ph.id_d2

-- Aggregation: detailed rows and subtotals by diagnosis and physician
GROUP BY ROLLUP (dg.diagnosis_description, ph.physician_name)

ORDER BY dg.diagnosis_description, ph.physician_name;

```

- **Technical Explanation:**

This query also compares physicians for the same diagnosis by looking at cost and workload. It joins the FACT_ENCOUNTER table with the dimension table for diagnosis (DIMENSION_5_DIAGNOSIS) and the physician dimension table (DIMENSION_2_PHYSICIAN). ROLLUP is used to generate aggregations at multiple levels for the query:

- cost and workload for each, (diagnosis, physician) pair,
- totals for each diagnosis across all physicians,
- the overall sum for all the diagnoses by every physician.

The query then calculates the SUM(f.total_cost) as total treatment cost, AVG(f.total_cost) as average treatment cost, and COUNT(*) as workload (the number of patients they admit).

The window function AVG(AVG(f.total_cost)) OVER (PARTITION BY diagnosis_description) calculates the overall average treatment cost for each diagnosis.

Finally, a CASE expression compares each doctor's average cost to the diagnosis-level average and labels them as Above, Below, or At the diagnostic cost.

- **Query 3: GROUPING SETS** - We are particularly interested in readmission rates - what percentage of patients are readmitted within 30 days of discharge? We want to see how these metrics trend over time (monthly and quarterly).
- **Business Question:** How does the 30-day readmission rate vary for each department by quarter, and what are the overall 30-day readmission rates per department (across all quarters), per quarter (across all departments), per month (across all departments), and for the whole hospital?
- **SQL Query:**


```

SELECT
-- Department dimension
d.department_name,

-- Time dimension: year is included to avoid mixing months/quarters across years
t.year,
t.quarter,
t.month,

-- Total number of encounters (admissions)
COUNT(*) AS total_admissions,

-- Number of encounters that are readmissions within 30 days
SUM(CASE WHEN f.readmission_flag = 'Y' THEN 1 ELSE 0 END) AS total_readmissions,

-- Readmission rate = (#readmissions / #admissions) * 100
ROUND(100 * SUM(CASE WHEN f.readmission_flag = 'Y' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(*), 0),
2) AS readmission_rate

FROM FACT_ENCOUNTER f
JOIN DIMENSION_3_DEPARTMENT d
  ON f.id_d3 = d.id_d3
JOIN DIMENSION_4_TIME t
  ON f.id_d4 = t.id_d4

GROUP BY GROUPING SETS (
-- 1) Department × Quarter trend (main analysis)
(d.department_name, t.year, t.quarter),

-- 2) Department × Month trend
(d.department_name, t.year, t.month),

-- 3) Department summary across all time
(d.department_name),

-- 4) Quarterly hospital-wide trend (all departments)
(t.year, t.quarter),

-- 5) Monthly hospital-wide trend (all departments)
(t.year, t.month),

-- 6) Grand total
()
)

ORDER BY
d.department_name NULLS LAST,
t.year,
t.quarter,
t.month;

```

- **Technical Explanation:**

This query analyzes the 30-day readmission rate for the hospital at various levels of detail based on the FACT_ENCOUNTER table with the DIMENSION_3_DEPARTMENT and DIMENSION_4_TIME dimension tables.

The main objective of this query is to demonstrate how the readmission behavior changes by department, by quarter, by month, and for the whole hospital.

The query uses the GROUPING SETS clause for this purpose, which provides multiple independent aggregation levels in a single query. The grouping sets will produce the following levels:

- department_name, year, quarter
- department_name, year, month
- department_name
- year, quarter
- year, month
- ()

Including t.year in the grouping sets ensures that months or quarters from different years are not merged together (being more logical and suitable with the real-life cases). In other words, the differences between Q1 2024 and Q1 2025 are maintained, which is important to preserve the accuracy of any time-series analysis.

The readmission rate is calculated by dividing the number of encounters with READMISSION_FLAG = 'Y' by the total number of encounters, times 100 to create a percent. Use NULLIF to prevent division-by-zero errors.

Notes: NULLS LAST makes the departmental rows appear first and the summary rows, which have NULL department, appear at the bottom; this way the results will be clearer and logical.

- **Query 4: ROLLUP** - We also track the total cost of treatment for each admission. We want to see how these metrics trend over time (monthly and quarterly).
- **Business Question:** How does the total treatment cost change for each department across different quarters, and what are the subtotal trends for each department and the hospital as a whole?
- **SQL Query:**

```
-- Department treatment cost trend by quarter using ROLLUP
SELECT
  -- Dimension: Department
  d.department_name,
  -- Dimension: Time (year + quarter to avoid mixing different years)
  t.year,
  t.quarter,

  -- Total treatment cost per group
  SUM(f.total_cost) AS total_treatment_cost,
```

```

-- Total admissions per group
COUNT(*) AS total_admissions,

-- Average treatment cost per admission
ROUND(SUM(f.total_cost) / NULLIF(COUNT(*), 0),
      2) AS average_cost_per_admission

FROM FACT_ENCOUNTER f
JOIN DIMENSION_3_DEPARTMENT d
  ON f.id_d3 = d.id_d3
JOIN DIMENSION_4_TIME t
  ON f.id_d4 = t.id_d4

-- Multi-level aggregation with ROLLUP:
-- (department, year, quarter)
-- (department, year)      -- subtotal per department-year
-- (department)           -- subtotal per department (all years)
-- ()                     -- grand total for the hospital
GROUP BY ROLLUP (
  d.department_name,
  t.year,
  t.quarter
)

ORDER BY
  d.department_name NULLS LAST,
  t.year,
  t.quarter;

```

- **Technical Explanation:**

This query analyzes total treatment cost and workload (number of admissions) at multiple levels for each department over time. It joins the FACT_ENCOUNTER table with the department dimension (DIMENSION_3_DEPARTMENT), and the time dimension (DIMENSION_4_TIME).

The main goal is to understand how costs change by department across different quarters; it also provides subtotals per department and an overall hospital-wide total.

The query uses the ROLLUP(d.department_name, t.year, t.quarter) operation in order to create multi-level aggregations in a single pass to observe total treatment cost, total admissions:

- department_name, year, quarter
- department_name, year
- department_name
- ()

The measures SUM(f.total_cost) and COUNT(*) calculate the total treatment cost and total admissions across each grouping level.

The derived metric average_cost_per_admission is the ratio of total cost to total admissions; here NULLIF is used in order to avoid a division by zero. This will give

further insight into the cost efficiency per admission for each department and timeframe.

The ORDER BY d.department_name NULLS LAST, t.year, t.quarter clause sorts the results so the detail rows for each department and quarter appear first, followed by department-level subtotals, ending with the grand total, which greatly enhances readability.

3.2 PL/SQL Functions

Function 1: get_age_group

```
-- PL/SQL function to convert a patient's birthdate and encounter date
-- into a categorical age group (Child, Adult, Middle-aged, Senior).
CREATE OR REPLACE FUNCTION get_age_group(
  -- Patient's date of birth
  p_birthdate IN DATE,
  -- Date of the encounter (from the Time dimension)
  p_full_date IN DATE
) RETURN VARCHAR2
IS
  -- Calculated age in full years at the encounter date
  v_age_years NUMBER;
  -- Resulting age group label to be returned by the function
  v_age_group VARCHAR2(20);
BEGIN
  -- Calculate age in full years at the encounter date.
  -- MONTHS_BETWEEN which is Built-in Oracle function, returns the number of months between two
  -- dates.
  -- Dividing by 12 converts months to years.
  -- FLOOR rounds the value down to the nearest whole number, removes the decimal part to get completed
  -- years only.
  v_age_years := FLOOR(MONTHS_BETWEEN(p_full_date, p_birthdate) / 12);

  -- Map the numeric age to a textual age group.
  -- This allows analyses to work with groups instead of raw ages.
  IF v_age_years < 18 THEN
    v_age_group := 'Child';
  ELSIF v_age_years BETWEEN 18 AND 44 THEN
    v_age_group := 'Adult';
  ELSIF v_age_years BETWEEN 45 AND 64 THEN
    v_age_group := 'Middle-aged';
  ELSE
    -- Age 65 and above is classified as Senior
    v_age_group := 'Senior';
  END IF;

  -- Return the age group label to the caller (SQL query)
  RETURN v_age_group;
END;
```

- **Explanation:** This function takes as input a patient's birthdate and the encounter date, calculates the age of the patient in full years at that encounter, and then further categorizes the age into one of four age groups: Child, Adult, Middle-aged, or Senior.

```

-- Analytical query using get_age_group to analyze average cost and
-- average length of stay by age group and department.
SELECT
  -- Use the custom PL/SQL function to derive an age group
  -- from the patient's birthdate and the encounter date.
  get_age_group(p.birthdate, t.full_date) AS age_group,

  -- Name of the department responsible for the encounter
  d.department_name,

  -- Average treatment cost per (age_group, department) combination
  AVG(f.total_cost)      AS average_total_cost,

  -- Average length of stay in days per (age_group, department) combination
  AVG(f.length_of_stay_days) AS average_length_of_stay

FROM FACT_ENCOUNTER f
  -- Join to the Patient dimension to access the birthdate
  JOIN DIMENSION_1_PATIENT  p ON f.id_d1 = p.id_d1
  -- Join to the Department dimension to know which department treated the patient
  JOIN DIMENSION_3_DEPARTMENT d ON f.id_d3 = d.id_d3
  -- Join to the Time dimension to get the full encounter date
  JOIN DIMENSION_4_TIME      t ON f.id_d4 = t.id_d4

GROUP BY
  get_age_group(p.birthdate, t.full_date),
  d.department_name;

```

- Explanation:** This query joins the fact table with the dimension table DIMENSION_1_PATIENT, DIMENSION_3_DEPARTMENT, and TIME and then utilizes the function get_age_group in the SELECT and GROUP BY clauses to group encounters by age group and department. It calculates average treatment cost and average length of stay for each age group/department combination. This will allow the hospital to observe how the costs and lengths of stay vary within age segments in different departments.

Function 2: get_readmission_risk

```

-- PL/SQL function that assigns a readmission risk label
-- based on three input factors: readmission flag, diagnosis severity, length of stay in days
CREATE OR REPLACE FUNCTION get_readmission_risk(
  p_readmission_flag IN CHAR,    -- input: 'Y' if patient was readmitted ('Y' or 'N')
  p_severity_level   IN VARCHAR2, -- input: severity classification of diagnosis ('High', 'Medium', 'Low')
  p_los              IN NUMBER    -- input: length of stay (number of days)
) RETURN VARCHAR2
IS
  -- Resulting risk label that will be returned
  v_risk_label VARCHAR2(30);
BEGIN
  -- Case 1:
  -- If the patient was readmitted AND had a High-severity diagnosis,
  -- they are considered the highest risk.
  IF p_readmission_flag = 'Y' AND p_severity_level = 'High' THEN

```

```

    v_risk_label := 'High risk';

-- Case 2:
-- If readmitted but severity is Medium or Low,
-- treat as Medium risk.
ELSIF p_readmission_flag = 'Y'
    AND p_severity_level IN ('Medium', 'Low') THEN
    v_risk_label := 'Medium risk';

-- Case 3:
-- If the patient was NOT readmitted, but:
-- stayed in the hospital for more than 10 days
-- and had a High severity diagnosis,
-- then classify as "Medium risk (long stay)".
ELSIF p_readmission_flag = 'N'
    AND p_los > 10
    AND p_severity_level = 'High' THEN
    v_risk_label := 'Medium risk (long stay)';

-- All other situations are considered low risk.
ELSE
    v_risk_label := 'Low risk';
END IF;

RETURN v_risk_label;
END;
/

```

- **Explanation:** This function takes in three inputs: the readmission flag, the severity level, and the length of stay in days. It returns a text label such as ‘High risk’, ‘Medium risk’, or ‘Low risk’. The underlying logic prioritizes actual readmissions with high severity as High risk, the remaining readmissions (with Medium or Low severity) as Medium risk, and also flags cases with no readmission but long stays combined with high severity as Medium risk (long stay), while everything else falls into Low risk.

```

SELECT
    d.department_name,
    get_readmission_risk(
        f.readmission_flag,
        dg.severity_level,
        f.length_of_stay_days
    ) AS risk_band,

    -- Count how many encounters fall into each (department, risk_band)
    COUNT(*) AS num_encounters

FROM FACT_ENCOUNTER f

-- Join to Department dimension to show department_name
JOIN DIMENSION_3_DEPARTMENT d
    ON f.id_d3 = d.id_d3

-- Join to Diagnosis dimension to access severity_level
JOIN DIMENSION_5_DIAGNOSIS dg
    ON f.id_d5 = dg.id_d5

-- Group by department and the function result
-- Detail rows: (department_name, risk_band)

```

```

-- Subtotals: (department_name, NULL)
-- Grand total: (NULL, NULL)
GROUP BY ROLLUP (
  d.department_name,
  get_readmission_risk(
    f.readmission_flag,
    dg.severity_level,
    f.length_of_stay_days
  )
);

```

- **Explanation:** This query joins the fact table FACT_ENCOUNTER with the dimensions DIMENSION_3_DEPARTMENT and DIMENSION_5_DIAGNOSIS to access each encounter's department and diagnosis severity level. Then, it applies the PL/SQL function get_readmission_risk in both the SELECT and GROUP BY ROLLUP clauses to classify every encounter into a readmission risk band based on the readmission flag, severity level, and length of stay.

Using the ROLLUP, the results are grouped by department_name and risk_band, providing detailed counts of encounters per department and risk category, subtotal counts per department, and a final grand total across the entire hospital.

This enables the hospital to see how many encounters fall into each of the risk levels within each department, identify departments with higher concentrations of high-risk or medium-risk cases, and inform quality-of-care monitoring and resource planning.

Function 3: compare_cost_with_baseprice

```

-- PL/SQL function to compare actual total_cost with the expected cost
-- based on service base_price and the number of services.
-- It returns the percentage difference between actual and baseline cost.
CREATE OR REPLACE FUNCTION compare_cost_with_baseprice(
  p_total_cost IN NUMBER, -- actual total cost from FACT_ENCOUNTER
  p_base_price IN NUMBER, -- base price per service from DIMENSION_6_SERVICE_CATALOG
  p_num_services IN NUMBER -- number of services for this encounter (from FACT_ENCOUNTER)
) RETURN NUMBER
IS
  v_baseline_cost NUMBER;
  -- Baseline cost represents the "expected" cost according to the service catalog.
  v_diff_percentage NUMBER;
  -- The percentage difference between actual total cost and the baseline cost.
  -- Positive = actual total cost is more expensive than baseline cost.
  -- Negative = actual total cost is cheaper than baseline cost.

BEGIN
  -- Compute the expected baseline cost:
  -- baseline = base_price * num_services
  v_baseline_cost := p_base_price * p_num_services;

  -- If the baseline cost is zero or NULL, avoid division by zero.
  -- In that case, just return NULL.
  IF v_baseline_cost IS NULL OR v_baseline_cost = 0 THEN
    RETURN NULL;
  END IF;

  -- Calculate percentage difference:

```

```

-- (actual - baseline) / baseline * 100
v_diff_percentage :=
  (p_total_cost - v_baseline_cost)
  / v_baseline_cost * 100;

RETURN v_diff_percentage;
END;
/

```

- Explanation:** This calculates how much more/less an encounter's actual treatment cost is, compared to a baseline cost derived from the service catalog. It computes the baseline as $\text{base_price} \times \text{num_services}$ and returns the percentage difference between the actual total cost and this baseline. Positive values indicate that the encounter was more expensive than expected; negative values indicate cheaper than expected. If the baseline is zero or missing, it returns NULL to avoid division by zero.

```

SELECT
  -- The department responsible for the encounter
  d.department_name,

  -- Quarter of the encounter (Q1, Q2, Q3, Q4)
  t.quarter,

  -- The physician who treated the patient
  ph.physician_name,

  -- Calculate the average percentage difference between
  -- actual total_cost and the baseline service cost.
  -- This uses the custom PL/SQL function compare_cost_with_baseprice.
  AVG(
    compare_cost_with_baseprice(
      f.total_cost, -- actual treatment cost from the fact table
      s.base_price, -- service base price from the service catalog
      f.num_services -- number of services used in this encounter
    )
  ) AS average_cost_difference_percentage
FROM FACT_ENCOUNTER f

-- Join the Department dimension to get department_name
JOIN DIMENSION_3_DEPARTMENT d
  ON f.id_d3 = d.id_d3

-- Join the Physician dimension to show which physician performed the encounter
JOIN DIMENSION_2_PHYSICIAN ph
  ON f.id_d2 = ph.id_d2

-- Join the Time dimension to know which quarter the encounter happened in
JOIN DIMENSION_4_TIME t
  ON f.id_d4 = t.id_d4

-- Join the Service Catalog to get the base price for each service
JOIN DIMENSION_6_SERVICE_CATALOG s
  ON f.id_d6 = s.id_d6

-- Group by all combinations of (department, quarter, physician)
GROUP BY CUBE (
  d.department_name,
  t.quarter,

```



```
ph.physician_name
);
```

- **Explanation:** This query joins the fact table FACT_ENCOUNTER with the Department, Physician, Time, and Service Catalog dimensions to access each encounter's department, quarter, physician, and service base price. It invokes in the SELECT clause a custom PL/SQL function compare_cost_with_baseprice to calculate, for each encounter, the percentage difference between the actual treatment cost and a baseline cost derived from the service catalog ($\text{base_price} \times \text{num_services}$). Using GROUP BY CUBE(department_name, quarter, physician_name), the query calculates an average cost difference percentage for all possible combinations of department, quarter, and physician, including subtotals and a grand total. It helps the hospital understand where actual treatment costs are consistently above or below the expected baseline across departments, quarters, and physicians.

Function 4:

```
-- PL/SQL function to classify a physician's workload
-- based on the number of encounters they handled.
CREATE OR REPLACE FUNCTION get_workload_band(
  p_encounter_count IN NUMBER -- total number of encounters for a physician
) RETURN VARCHAR2
IS
  -- Text label describing the workload band
  v_workload_band VARCHAR2(20);
BEGIN
  -- If the physician handled 10 or fewer encounters → Low workload
  IF p_encounter_count <= 10 THEN
    v_workload_band := 'Low workload';

  -- If the physician handled between 11 and 20 encounters → Medium workload
  ELSIF p_encounter_count BETWEEN 11 AND 20 THEN
    v_workload_band := 'Medium workload';

  -- If the physician handled more than 20 encounters → High workload
  ELSE
    v_workload_band := 'High workload';
  END IF;

  -- Return the workload band label
  RETURN v_workload_band;
END;
```

- **Explanation:** This function takes a single numeric input, p_encounter_count, referring to the number of encounters handled by a physician. Conditional logic (IF / ELSIF / ELSE) is used to map that numeric count into one of three workload bands: 'Low workload', 'Medium workload', or 'High workload'. This will make it easy to categorize physicians by workload intensity for reporting and analysis.

```
SELECT
  -- Classify physicians into workload bands using the function
  get_workload_band(encounters.encounters_count) AS workload_band,
```

```

-- Number of physicians in this workload band
COUNT(*) AS number_of_physicians,

-- Total encounters handled by all physicians in this band
SUM(encounters.encounters_count) AS total_encounters
FROM (
  -- Subquery: count encounters per physician
  SELECT
    ph.physician_name,
    COUNT(*) AS encounters_count  -- total encounters for this physician
  FROM FACT_ENCOUNTER f
  JOIN DIMENSION_2_PHYSICIAN ph
    ON f.id_d2 = ph.id_d2
  GROUP BY ph.physician_name
) encounters
GROUP BY
  get_workload_band(encounters.encounters_count);

```

- Explanation:** This query counts the number of encounters handled by each physician in a subquery (encounters). Each line in the subquery consists of one physician and the total number of encounters. In the outer query, the custom PL/SQL function `get_workload_band` is used to take this count and map the physicians into workload bands such as 'Low workload', 'Medium workload', and 'High workload'. It then groups the result by workload band to calculate the number of physicians in each category `number_of_physicians` and the total number of encounters in each band for the concerned physicians `total_encounters`. This gives a clear overview of the distribution of the workload of the physicians throughout the hospital.

3.3 Advanced Queries (using Functions, MODEL, etc.)

Query 1: For each department and month, what are the current readmission rate and total treatment cost, and how would the number of admissions and the total cost change if we reduced readmissions by 10%?

- Business Question:** For each department and month, what are the current readmission rate and total treatment cost, and how would the number of admissions and the total cost change if we reduced readmissions by 10%?
- SQL Query:**

```

-- Readmission reduction scenario using MODEL
SELECT
  -- Dimensions and measures returned to the user
  department_name,
  month_number,
  year_number,
  total_admissions,
  total_readmissions,
  total_treatment_cost,
  readmission_rate,
  scenario_readmissions,
  scenario_total_admissions,
  scenario_total_cost
FROM (

```

```

-- Aggregate data at the Department-Month-Year level
SELECT
  d.department_name AS department_name, -- department (e.g., Cardiology)
  t.month AS month_number, -- month number from time dimension
  t.year AS year_number, -- year from time dimension
  COUNT(*) AS total_admissions, -- total number of admissions in this dept-month-year
  SUM(CASE
    WHEN f.readmission_flag = 'Y' -- if encounter is a readmission
    THEN 1
    ELSE 0
  END) AS total_readmissions, -- total number of readmissions
  SUM(f.total_cost) AS total_treatment_cost -- total treatment cost for this dept-month-year
FROM FACT_ENCOUNTER f
JOIN DIMENSION_3_DEPARTMENT d
  ON f.id_d3 = d.id_d3
JOIN DIMENSION_4_TIME t
  ON f.id_d4 = t.id_d4
GROUP BY
  d.department_name,
  t.year,
  t.month
)
-- Use MODEL to do spreadsheet-like what-if calculations
MODEL
  -- PARTITION BY: each year is treated as a separate "sheet"
  PARTITION BY (year_number)

  -- DIMENSION BY: rows and columns of the "sheet"
  -- - department_name: row identifier
  -- - month_number: column identifier (within a year)
  DIMENSION BY (department_name, month_number)

  -- MEASURES: numeric values stored in each cell of the sheet
  MEASURES (
    total_admissions, -- base measure: total admissions
    total_readmissions, -- base measure: total readmissions
    total_treatment_cost, -- base measure: total cost

    -- derived measures (initially NULL; will be filled by RULES)
    CAST(NULL AS NUMBER) AS readmission_rate, -- current readmission rate
    CAST(NULL AS NUMBER) AS scenario_readmissions, -- readmissions after a 10% reduction
    CAST(NULL AS NUMBER) AS scenario_total_admissions, -- total admissions in the scenario
    CAST(NULL AS NUMBER) AS scenario_total_cost -- total cost in the scenario
  )

  -- RULES: formulas that compute the derived measures for each cell
  RULES (
    -- Rule 1: Current readmission rate for each department-month-year
    -- readmission_rate = total_readmissions / total_admissions
    -- NVL(..., 0) avoids NULL when there are no readmissions
    -- NULLIF(..., 0) avoids division by zero when there are no admissions
    readmission_rate[ANY, ANY] =
      NVL(total_readmissions[CV(department_name), CV(month_number)], 0)
      / NULLIF(total_admissions[CV(department_name), CV(month_number)], 0),

    -- Rule 2: Scenario – reduce readmissions by 10%
    -- scenario_readmissions = 90% of current readmissions
    scenario_readmissions[ANY, ANY] = TRUNC(total_readmissions[CV(department_name),
CV(month_number)] * 0.9),

```

```

-- Rule 3: New total admissions after reducing readmissions
-- We assume each avoided readmission removes one admission from the total
-- reduction = total_readmissions - scenario_readmissions
-- scenario_total_admissions = total_admissions - reduction
scenario_total_admissions[ANY, ANY] =
    total_admissions[CV(department_name), CV(month_number)]
    - (total_readmissions[CV(department_name), CV(month_number)]
      - scenario_readmissions[CV(department_name), CV(month_number)]),

-- Rule 4: New total cost in the scenario
-- Assume each avoided readmission saves 500 cost units
-- reduction = total_readmissions - scenario_readmissions
-- scenario_total_cost = total_treatment_cost - reduction * 500
scenario_total_cost[ANY, ANY] =
    total_treatment_cost[CV(department_name), CV(month_number)]
    - (total_readmissions[CV(department_name), CV(month_number)]
      - scenario_readmissions[CV(department_name), CV(month_number)]) * 500
);

```

- **Technical Explanation:**

This query performs a department–month–year aggregation of hospital encounters and then applies the Oracle MODEL clause to run a spreadsheet-style simulation directly on the summarized warehouse data. The inner SELECT joins the fact table with the department and time dimensions to compute three base measures for each department and month: total admissions, total readmissions, and total treatment cost. The MODEL clause then treats these results as a multidimensional grid, where PARTITION BY (year_number) creates a separate calculation sheet for each year, and DIMENSION BY (department_name, month_number) defines each grid cell. Within this model, base measures are loaded as-is, while derived measures such as readmission rate and scenario values are declared using CAST(NULL AS NUMBER) to ensure correct numeric typing. The RULES section encodes business logic: Rule 1 computes the current readmission rate using NVL and NULLIF to handle missing data and avoid division by zero; Rule 2 simulates a 10% reduction in readmissions and uses TRUNC to keep the result as an integer; Rule 3 recalculates total admissions after removing avoided readmissions; and Rule 4 updates total cost assuming each prevented readmission saves 500 units. This demonstrates a non-trivial use of the MODEL clause for what-if scenario analysis in a healthcare warehouse.

4. Task 4: Aggregate Table

- **Justification:**

The metric we chose for aggregation is "total monthly treatment cost and total admissions per department". We chose this because hospital management is highly interested in monitoring department performance over time, e.g., monthly cost and workload. Running this analysis directly on the FACT_ENCOUNTER table requires scanning all encounter records, joining with the department and time dimensions and performing GROUP BY aggregations on (department, year, month).

In a real data warehouse, the fact table would contain hundreds of thousands or millions of rows, and these kinds of queries are very slow and resource intensive.

To optimize for this, we designed an aggregate table AGG_DEPT_MONTHLY_STATS that pre-calculates and stores:

- total monthly admissions per department
- total monthly treatment cost per department
- average cost per admission per department per month

This table uses the surrogate key id_d3, department key, along with year and month for its grain. This design:

- stays consistent with our star schema, keeps surrogate keys instead of textual names,
- reduces query time by scanning a very small pre-aggregated table.
- and still allows us to join back to DIMENSION_3_DEPARTMENT when needed.

- **DDL for Aggregate Table:**

```
-- Monthly department performance summary
CREATE TABLE AGG_DEPT_MONTHLY_STATS (
  id_d3          NUMBER NOT NULL,    -- Department surrogate key
  year           NUMBER NOT NULL,    -- Year from DIMENSION_4_TIME
  month          NUMBER NOT NULL,    -- Month from DIMENSION_4_TIME

  total_admissions  NUMBER,          -- Total number of encounters in the month
  total_treatment_cost NUMBER,       -- Sum of total_cost for the month
  avg_cost_per_admission NUMBER,     -- Average cost per admission for the month

  -- Composite primary key: each (department, year, month) is unique
  CONSTRAINT pk_agg_dept_month PRIMARY KEY (id_d3, year, month)
);

-- It guarantees that id_d3 in the aggregate table always matches a real department.
ALTER TABLE AGG_DEPT_MONTHLY_STATS
ADD CONSTRAINT fk_agg_dept_month_dept
  FOREIGN KEY (id_d3)
  REFERENCES DIMENSION_3_DEPARTMENT(id_d3);
```

Explanation:

The AGG_DEPT_MONTHLY_STATS table is our aggregate table, built to speed up the queries for the analytical dashboard. Rather than calculating the totals and averages from the very large fact table each time a user wants to see a report for one month, we pre-calculate these values and store them here. Each row represents one department in one year and one month. The table includes the surrogate key of the department id_d3 and time attributes year, month. For each combination of department and month, we store the total number of admissions, the total treatment cost, and the average cost per admission.

Having this table allows the data warehouse to avoid repeatedly scanning and grouping the large FACT_ENCOUNTER table. This is of immense use when building dashboards, as there are constant requests for monthly trends that must load instantly. The composite primary key id_d3, year, month ensures that each department can only appear once per month without duplication, maintaining clean aggregate data.

- **Sample Query (Fast):**

```
-- Fast dashboard query using the aggregate table
SELECT
    d.department_name,      -- department name from dimension table
    a.year,                 -- aggregated year
    a.month,                -- aggregated month
    a.total_treatment_cost, -- pre-calculated total cost
    a.total_admissions,     -- pre-calculated admissions
    a.avg_cost_per_admission -- pre-calculated average cost
FROM AGG_DEPT_MONTHLY_STATS a
JOIN DIMENSION_3_DEPARTMENT d
    ON a.id_d3 = d.id_d3
ORDER BY d.department_name, a.year, a.month;
```

Explanation:

This is a query optimized for fast dashboard reporting; instead of scanning and aggregating the large fact table, it retrieves pre-computed monthly statistics directly from the AGG_DEPT_MONTHLY_STATS table. A simple join with the DIMENSION_3_DEPARTMENT table brings in the department name so the results are readable.

Since one row per department-month combination already exists in the aggregate table, the query simply reads those values and sorts them. This makes for a very lightweight query that is suitable to run multiple times in dashboards or reports. It shows, for each department and month, the total number of admissions, total treatment cost, and average cost per admission-all without performing expensive aggregations.

- **Population Procedure:**

```
CREATE OR REPLACE PROCEDURE populate_agg_dept_monthly AS
BEGIN
    -- Empty the aggregate table before repopulating
    EXECUTE IMMEDIATE 'TRUNCATE TABLE AGG_DEPT_MONTHLY_STATS';

    -- Insert summarized data from the fact table
    -- Grouping by department + year + month
    INSERT INTO AGG_DEPT_MONTHLY_STATS (
        id_d3,
        year,
        month,
        total_admissions,
        total_treatment_cost,
        avg_cost_per_admission
    )
    SELECT
        f.id_d3,          -- department surrogate key
        t.year,          -- year from time dimension
        t.month,         -- month from time dimension
        COUNT(*) AS total_admissions, -- number of encounters
        SUM(f.total_cost) AS total_treatment_cost, -- total cost
        ROUND(AVG(f.total_cost), 2) AS avg_cost_per_admission -- avg cost
    FROM FACT_ENCOUNTER f
    JOIN DIMENSION_4_TIME t
        ON f.id_d4 = t.id_d4 -- linking encounter to time dimension
    GROUP BY
        f.id_d3,
        t.year,
        t.month;          -- one row per department, year and month

    COMMIT;
END;

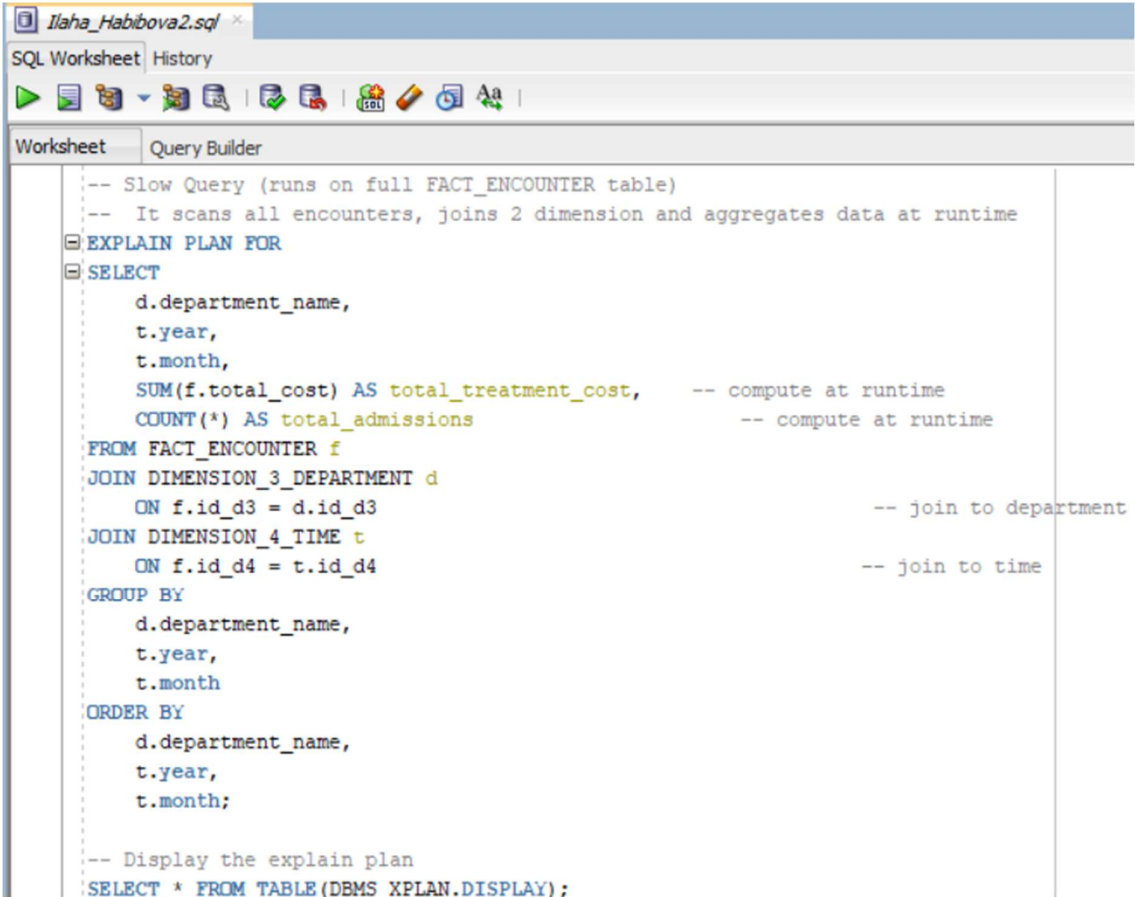
/
```

```
EXEC populate_agg_dept_monthly;
```

Explanation:

Procedure `populate_agg_dept_monthly` automatically rebuilds the whole aggregate table. First, it truncates (`TRUNCATE TABLE`) the old content to make sure the table always contains fresh and accurate summary data. Then it inserts new rows by selecting from `FACT_ENCOUNTER` and grouping the records by department, year, and month. During this process, it calculates three important analytical measures: `total_admissions` = the number of encounters for the department that month `total_treatment_cost` = total cost of all encounters `avg_cost_per_admission` = average cost for a single encounter This will pre-calculate all month values in advance and store them in a compact and ready-to-use manner. This procedure can be rerun every time there is an insert or update of fact data to refresh the aggregate table. The `COMMIT` statement at the end confirms that all computed rows are saved. In practice, such a procedure is essential in large data warehouses because, by moving heavy computations to the ETL stage, the analytical queries can then be executed extremely fast.

- **Performance EXPLAIN PLAN Analysis:**



```
-- Slow Query (runs on full FACT_ENCOUNTER table)
-- It scans all encounters, joins 2 dimension and aggregates data at runtime
EXPLAIN PLAN FOR
SELECT
    d.department_name,
    t.year,
    t.month,
    SUM(f.total_cost) AS total_treatment_cost,    -- compute at runtime
    COUNT(*) AS total_admissions                -- compute at runtime
FROM FACT_ENCOUNTER f
JOIN DIMENSION_3_DEPARTMENT d
    ON f.id_d3 = d.id_d3                        -- join to department
JOIN DIMENSION_4_TIME t
    ON f.id_d4 = t.id_d4                        -- join to time
GROUP BY
    d.department_name,
    t.year,
    t.month
ORDER BY
    d.department_name,
    t.year,
    t.month;

-- Display the explain plan
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Result:

Script Output x Query Result x

SQL | All Rows Fetched: 22 in 0.156 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 3997496984

2

3 -----

4 | Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |

5 -----

6 | 0 | SELECT STATEMENT | | 15 | 540 | 10 (20) | 00:00:01 |

7 | 1 | SORT GROUP BY | | 15 | 540 | 10 (20) | 00:00:01 |

8 |* 2 | HASH JOIN | | 60 | 2160 | 9 (12) | 00:00:01 |

9 | 3 | MERGE JOIN | | 60 | 1560 | 6 (17) | 00:00:01 |

10 | 4 | TABLE ACCESS BY INDEX ROWID | DIMENSION_3_DEPARTMENT | 6 | 96 | 2 (0) | 00:00:01 |

11 | 5 | INDEX FULL SCAN | SYS_C00666479 | 6 | | 1 (0) | 00:00:01 |

12 |* 6 | SORT JOIN | | 60 | 600 | 4 (25) | 00:00:01 |

13 | 7 | TABLE ACCESS FULL | FACT_ENCOUNTER | 60 | 600 | 3 (0) | 00:00:01 |

14 | 8 | TABLE ACCESS FULL | DIMENSION_4_TIME | 10 | 100 | 3 (0) | 00:00:01 |

15 -----

16

17 Predicate Information (identified by operation id):

18 -----

19

20 2 - access("F"."ID_D4"="T"."ID_D4")

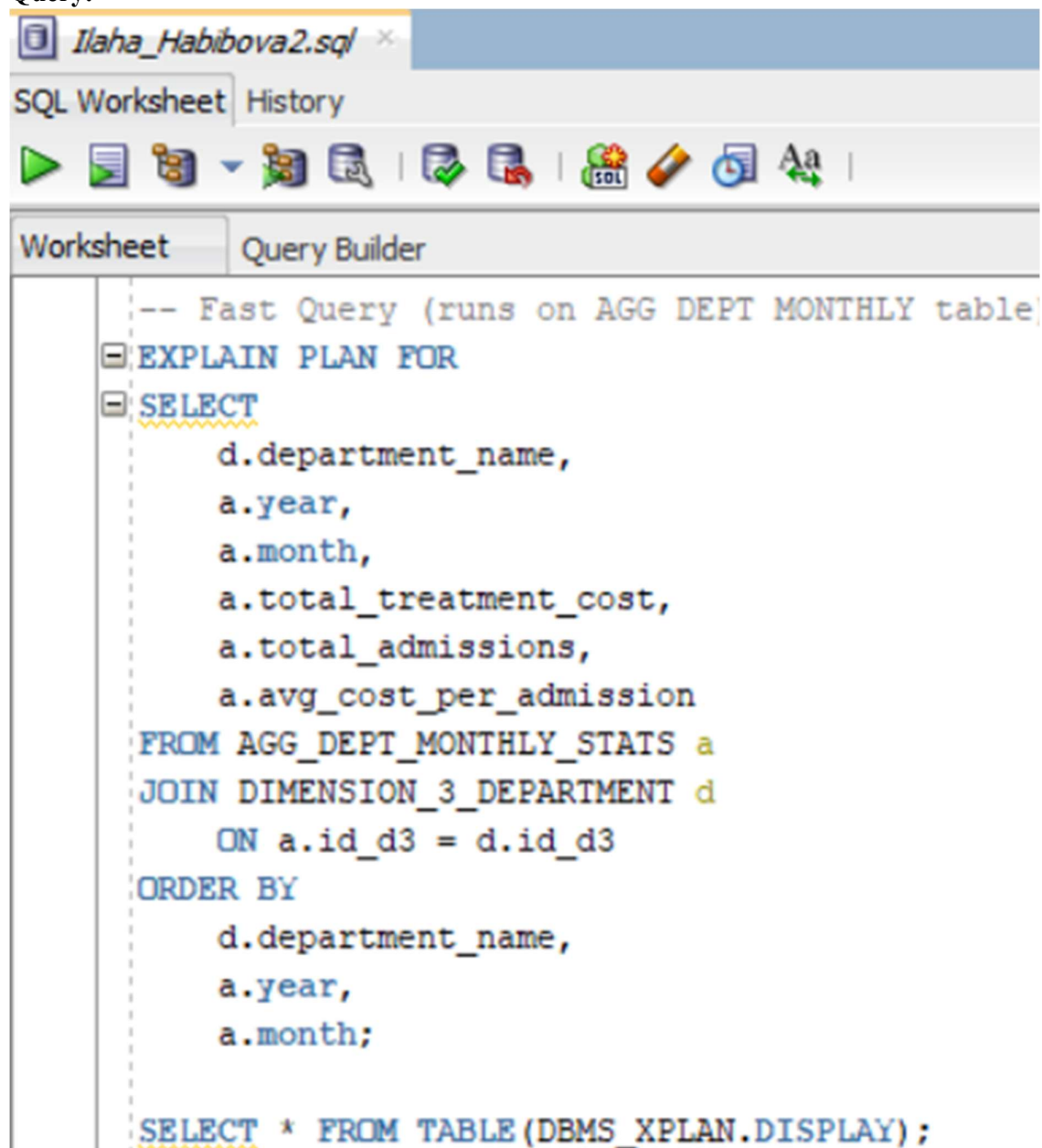
21 6 - access("F"."ID_D3"="D"."ID_D3")

22 filter("F"."ID_D3"="D"."ID_D3")

Explanation

This query operates on the huge FACT_ENCOUNTER table: it needs to read all the encounter rows, join with the two dimension tables, and compute SUM(total_cost) and COUNT(*) for each department, year, and month. In other words, every time this query runs, the database has to do a full table scan, several joins, and a GROUP BY. Such queries are common in data analytics but can become very slow as the fact table grows. For an actual production environment with millions of encounters, this would be expensive in terms of time and resources. This slow query represents the “baseline performance” before using an aggregate table, and is used for comparison in the EXPLAIN PLAN analysis.

Query:

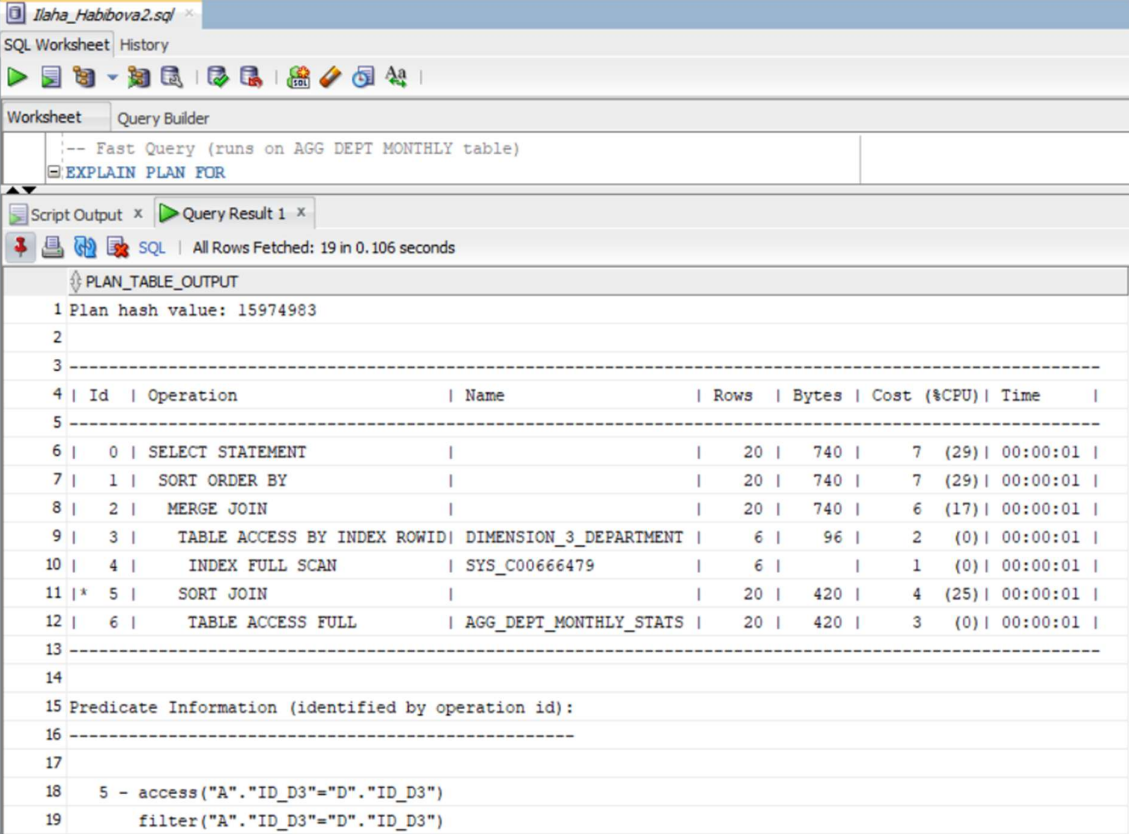


The screenshot shows an SQL Worksheet window titled 'Ilaha_Habibova2.sql'. It has tabs for 'SQL Worksheet' and 'History'. Below the tabs is a toolbar with icons for running queries, saving, and other database functions. The main area is divided into 'Worksheet' and 'Query Builder' tabs. The 'Worksheet' tab is active, displaying the following SQL query:

```
-- Fast Query (runs on AGG DEPT MONTHLY table)
EXPLAIN PLAN FOR
SELECT
    d.department_name,
    a.year,
    a.month,
    a.total_treatment_cost,
    a.total_admissions,
    a.avg_cost_per_admission
FROM AGG_DEPT_MONTHLY_STATS a
JOIN DIMENSION_3_DEPARTMENT d
    ON a.id_d3 = d.id_d3
ORDER BY
    d.department_name,
    a.year,
    a.month;

SELECT * FROM TABLE (DBMS_XPLAN.DISPLAY);
```

Result:



SQL Worksheet History

Worksheet Query Builder

```
-- Fast Query (runs on AGG_DEPT_MONTHLY table)  
EXPLAIN PLAN FOR
```

Script Output x Query Result 1 x

SQL | All Rows Fetched: 19 in 0.106 seconds

PLAN_TABLE_OUTPUT							
1 Plan hash value: 15974983							
2							
3 -----							
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
5 -----							
6	0	SELECT STATEMENT		20	740	7 (29)	00:00:01
7	1	SORT ORDER BY		20	740	7 (29)	00:00:01
8	2	MERGE JOIN		20	740	6 (17)	00:00:01
9	3	TABLE ACCESS BY INDEX ROWID	DIMENSION_3_DEPARTMENT	6	96	2 (0)	00:00:01
10	4	INDEX FULL SCAN	SYS_C00666479	6		1 (0)	00:00:01
11	5	SORT JOIN		20	420	4 (25)	00:00:01
12	6	TABLE ACCESS FULL	AGG_DEPT_MONTHLY_STATS	20	420	3 (0)	00:00:01
13 -----							
14							
15 Predicate Information (identified by operation id):							
16 -----							
17							
18	5	- access("A"."ID_D3"="D"."ID_D3")					
19		filter("A"."ID_D3"="D"."ID_D3")					

Explanation:

By contrast, the EXPLAIN PLAN of the fast query reveals a dramatically simpler and cheaper execution path: it scans only the small AGG_DEPT_MONTHLY_STATS table and performs one join to fetch department names. There is no GROUP BY, no aggregation, and no heavy sorting. The difference in estimated cost clearly shows the performance benefit of maintaining an aggregate table. By moving the computation to ETL time, the dashboards and analytical queries run much faster, as the fact table grows in size. In fact, this is the very practical purpose of aggregate tables in real-world data warehouses: improving query speed and overall responsiveness of the system.

Analysis:

EXPLAIN PLAN for the slow query indicates that Oracle needs to read all rows from FACT_ENCOUNTER, join them with DIMENSION_3_DEPARTMENT and DIMENSION_4_TIME, and then perform a GROUP BY on (department_name, year, month). Realistically, there must be thousands of encounters; hence, this would turn out to be an expensive operation with high estimated cost. The plan for the fast query on AGG_DEPT_MONTHLY_STATS is much simpler. The database only has to scan the small aggregate table—one row per department–year–month—and perform a lightweight join to DIMENSION_3_DEPARTMENT to fetch the department name. Since no grouping or aggregation is required at query time, the estimated cost is much lower. This demonstrates the

main advantage of the aggregate table: it shifts the heavy computation from query time to ETL time. Dashboards and recurring monthly/department reports can now be run from a compact, pre-aggregated dataset, greatly enhancing responsiveness and scalability as the fact table grows.

5. Demonstration Video Link

A link to our 5-7 minute group video demonstration:

Link:

Google Drive:

<https://drive.google.com/file/d/1arB0AXSZxO2kUMYfX53WoaC9h6VdfWRU/view?usp=sharing>

Youtube:

<https://youtu.be/PEi8mJsZGVY?si=2-9fcdkhmUiSs7Rf>

6. Group Work Report

Overall, our workflow was efficient, supportive, and well-coordinated, which allowed us to complete the project successfully.

Work Division:

Since we are only two people, we easily divided the tasks according to their points equally. The student Ngoc Bao Tram Tran worked mainly on Task 2 and Task 3. The student Ilaha Habibova worked on Task 1 and Task 4. Once Ilaha finished Task 1, Tram checked her star schema design and made a few additions for the Task 2 and Task 3 requirements. The two also checked each other's tasks upon completion and made corrections when needed. Each person also filled their parts in report accordingly. Because the tasks allocated to Ilaha had more points, Tram had to combine the code from both members into one work file (txt) and was responsible for the Group Work Report: including content and format (a brief reflection on your group's process - how you divided the work, what went well and what didn't, and what challenges you faced) and the Conclusions section (a summary of the work, discussing the most interesting or challenging design and implementation details). However, Ilaha also wrote the parts like "What challenges did you face?" and "the most interesting or challenging design and implementation details," then gave feedback and made some modifications to other parts of Group Work Report as well.

Process:

What went well in your group collaboration?

To address what worked well in our group work, whenever there were instances where we encountered issues which we did not grasp properly or know the best way of solving them, we collectively discussed them and helped each other comprehend them. In addition to this, if there was something that did not entirely complement the work of the other person in our group that one of us observed, suggestions were raised.

The allocation of tasks was beneficial because each of us was able to work independently on a task that added to the whole. Our communication was good, and we regularly had discussions about schema design issues such as surrogate key columns, type 2 Slowly Changing Dimensions, dimension table attributes and fact table measures to maintain consistency across all tasks.

Reviewing each other's work helped us catch mistakes early.

What challenges did you face, and how did you overcome them?

As far as challenges are concerned, Tram has the tendency to overthink things and at times tries to do things in a more complex manner than required. That's why Ilaha had to remind her that our solution should be feasible to implement during that practical work. To begin with Tram also had problems understanding the project description and the way she should do things. That's why she had to rely on presentations of the lectures she had attended and internet websites to understand the description.

One challenge Tram faced was aligning the design from Task 1 with the requirements of Task 2 and Task 3. Therefore, Tram made some additions, but Ilaha told her, she must remove certain attributes (like age_group as an attribute in the DIMENSION_1_PATIENT table), because they must be implemented as functions. We (both of us – Tram and Ilaha) need to ensure compatibility across tasks.

Another challenge was handling the complexity of writing non-trivial analytical queries, especially those involving CUBE/ROLLUP, window functions, and the MODEL clause, which required additional time for debugging and fully satisfied the assignment rubric.

In the beginning, Ilaha also found it difficult to understand EXPLAIN PLAN results and interpret them. Therefore, she checked different tutorials on websites and Youtube videos.

7. Conclusions

The project work involved the conceptualization and development of a data warehouse system concerning patient admissions at hospitals. Throughout the process, we created a star schema, built dimension and fact tables, loaded data, and developed analytical SQL queries that employ ROLLUP, CUBE, GROUPING SETS, window functions, and PL/SQL user-defined functions. The data ware house helps in analysing patient flows, physicians' workloads, department's performance, and readmission rates.

- **What are the key takeaways from this project?**

- A good star schema simplifies complex hospital data.
- The fact table design (ENCOUNTER) is a foundation for analysis. Define a clear grain for the fact table is really important.
- Design clear dimensions (Patient, Physician, Department, Diagnosis, Service, Time) allowed us to analyze admissions from many perspectives without writing complex query.
- We also learned how surrogate keys and SCD Type 2 ensure historical accuracy - a critical requirement for healthcare analytics. For example, tracking changes in patient insurance over time helps avoid inaccurate historical reporting.
- Establishing one encounter = one fact row helped guide all subsequent design decisions, including measures, dimension keys, and SCD logic.
- From an implementation perspective, building a PL/SQL data-loading procedure demonstrated the value of automating ETL for data populating consistency and repeatability.
- Writing complex analytical queries showed how warehouse structures directly support business questions such as readmission trends, quarterly cost patterns, and physician performance. Advanced SQL analytics greatly improved insight quality.
- Finally, the aggregate table reinforced how pre-aggregation improves query speed, which is essential for real-time dashboards.

- **What was the most challenging or interesting part? (e.g., "The most interesting part was seeing the EXPLAIN PLAN cost drop after implementing the aggregate table...")**

The most intriguing part of this project was how our analysis became that much clearer and quicker once the design of the data warehouse was set correctly.

That made quite a difference, especially when we implemented the AGG_DEPT_MONTHLY_STATS table: the EXPLAIN PLAN clearly showed how pre-aggregating department - month statistics turned a heavy GROUP BY query into much faster and simpler. It is satisfying to see how a well-planned aggregate table can dramatically improve performance.

Another highlight was the analytic query that compared physicians within each diagnosis category. That query combined ROLLUP with window functions and involved nested expressions such as AVG(AVG(f.total_cost)) OVER (PARTITION BY dg.diagnosis_description) to calculate the diagnosis-level averages; from there, we knew we could determine the Above, Below, or At the diagnosis average cost for each physician. The design of this query was rather tricky because it relied on carefully controlling grouping levels, being mindful of how window functions would behave on aggregated data, and ensuring that subtotals generated by ROLLUP still made sense.

The load_fact_encounter procedure was also both difficult and interesting to build. Instead of generating meaningless random data, we aimed to simulate realistic hospital

behavior: each doctor belongs only to their appropriate department, departments are linked to plausible diagnoses, and each department uses a realistic subset of medical services. On top of that, measures like `total_cost`, `length_of_stay_days`, and `readmission_flag` were generated with controlled randomness to mimic real clinical variation. Balancing correctness, randomness, SCD validity periods, and complex join logic inside a single ETL procedure made this one of the most challenging implementation tasks, but it resulted in a much believable dataset.

Besides that, there were several other meaningful examples during the project. It became easier to write advanced CUBE queries when the Time dimension was clearly structured with year, quarter, and month. Also, implementing SCD Type 2 for the Patient dimension was equally challenging but worth doing, since it made sure historical analyses remained accurate if a patient's insurance changed later. Creating custom PL/SQL functions made us think more like real data engineers, especially in creating logics that classified workload bands or cost categories. It felt very satisfying when the more advanced queries, like physician comparisons using window functions, started to show clear, interpretable patterns in the data. Taken together, these elements make for a technically challenging yet very rewarding project; it shows how meaningful insights can be unlocked from complex hospital data with thoughtful data warehouse design.

- **What insights did your data warehouse provide about your domain?**

The completed warehouse offered several insights into the hospital domain. We observed how average length of stay and treatment cost vary significantly across departments and diagnoses, providing valuable information for resource planning. Readmission analysis, which is one of the core goals described in the domain specification, became much more interpretable when aggregated by month, quarter, and department. Physician-level queries bring out variations in workload and treatment cost behavior, thus showing opportunities for operational balancing or performance evaluation. Age-group analytics-possible thanks to our PL/SQL function-offer insights relevant to budgeting and care-quality monitoring. Scenario modeling using the MODEL clause showed how a 10% reduction in readmissions could decrease admissions and treatment costs, and thus the value of predictive and "what-if" analytics in making hospital decisions.

In summary, this project enabled us to translate a real-world healthcare scenario into a robust analytical system. It further strengthened our understanding of various concepts in data warehousing-schema design, ETL pipelines, OLAP queries, PL/SQL functions, SCD management, and performance optimization-and how these components work together to enable informed, data-driven decision-making in a hospital environment.